

Servidores HTTP y Express

GUÍA DE CONCEPTOS DETALLADOS - UNIDAD 2

1. Servidor HTTP con Node y Módulos Nativos

Antes de utilizar frameworks, Node.js permite crear servidores y manejar archivos utilizando sus módulos nativos integrados.

Módulos Esenciales

- **Módulo `fs` (File System):** Permite leer (`fs.readFile`), escribir (`fs.writeFile`) y administrar archivos y directorios de forma nativa.
- **Módulo `path` :** Fundamental para trabajar con rutas de archivos de manera segura en cualquier sistema operativo (ej. `path.join()` , `path.extname()`).

Para crear un servidor puro, se utiliza el módulo nativo `http` mediante `http.createServer()` , configurando manualmente las cabeceras (headers) y el ruteo mediante condicionales `if/else` evaluando la URL solicitada.

2. Introducción a Express

Express es un framework web minimalista y flexible para Node.js que simplifica enormemente la creación de aplicaciones web y APIs.

Ventajas sobre HTTP nativo:

- Manejo de rutas simplificado y elegante.
- Sistema integrado de Middleware.
- Mejor manejo de errores y códigos de estado.
- Arquitectura escalable y fácil de mantener.

3. Ruteo Básico en Express

Express define rutas asociándolas directamente a los **Métodos HTTP** (GET, POST, PUT, DELETE). En cada ruta interactuamos con dos objetos fundamentales:

El Objeto Request (<code>req</code>)	El Objeto Response (<code>res</code>)
Contiene la información de la petición que hace el cliente al servidor.	Contiene los métodos para responderle al cliente.
• <code>req.params</code> : Variables en la URL (ej. /usuario/:id). • <code>req.query</code> : Parámetros tras el signo ? (ej. ?categoria=pc). • <code>req.body</code> : Datos en el cuerpo de la petición (JSON/Forms).	• <code>res.send()</code> : Envía texto o HTML. • <code>res.json()</code> : Envía un objeto formato JSON. • <code>res.status(200)</code> : Define el código de estado HTTP.

4. Middlewares: El Corazón de Express

Los **Middlewares** son funciones que se interceptan en el medio del ciclo de solicitud/respuesta. Tienen acceso al objeto `req`, al objeto `res`, y a la función `next()`, que delega el control a la siguiente función.

Tipos de Middleware

1. **De nivel de aplicación:** Se ejecutan en todas las peticiones (ej. `app.use()` para loguear datos).
2. **De nivel de ruta:** Se aplican solo a rutas o conjuntos de rutas específicas (ej. validar si un usuario existe antes de mostrar su perfil).
3. **De manejo de errores:** Capturan excepciones en la aplicación (usan 4 parámetros: `err, req, res, next`).
4. **De terceros:** Paquetes externos instalados vía NPM, como `morgan` (logs) o `cors`.

5. Rutas Avanzadas y Modularidad

Organización de Rutas (Router)

A medida que la aplicación crece, definir todas las rutas en un solo archivo es insostenible. Express provee `express.Router()` para crear módulos de rutas separadas (ej. un archivo para *usuarios*, otro para *productos*) y luego "montarlos" en el archivo principal usando `app.use('/usuarios', usuariosRouter)`.

Rutas Paramétricas y Expresiones Regulares

Las rutas pueden ser dinámicas definiendo parámetros con dos puntos (`:`):

```
// Ruta con múltiples parámetros
app.get('/productos/:categoria/:id', (req, res) => {
  const categoria = req.params.categoria;
  const id = req.params.id;
  res.send(`Producto ${id} de la categoría ${categoria}`);
});
```

Express también permite el uso de expresiones regulares para coincidencias de URL más complejas (ej. rutas que terminen obligatoriamente en `.html`).

Middlewares Avanzados

- **Validación de Parámetros:** Middlewares diseñados específicamente para verificar que un ID sea numérico o válido antes de ejecutar la lógica de la ruta.
- **CORS:** Permite (o restringe) que dominios externos consuman nuestra API.
- **Archivos Estáticos:** Utilizando el middleware integrado `express.static('public')`, podemos servir directamente imágenes, CSS o HTML al cliente sin programar rutas manualmente para cada archivo.